

Threads ([Previous Notes](#))

Introduction

Initial comments

- The goal for this presentation will be to understand the concept of threads and how to write programs using threads.
- We will use the POSIX standard threads in this course.
- There are other implementations for threads. Windows has them and so does C++ . These will be similar. **What are some pros and cons of using Linux threads rather than those of C++?**

POSIX is a standard for operating systems. All UNIX systems that I know of support at least one level of this standard. So, if we learn POSIX threads the code we write will be reasonably portable among all systems that support this standard . (Of course C++ threads are more portable) Note: POSIX is a product of the IEEE. Note: once you are comfortable using threads on one system, it is not that difficult to learn them on another system.

The what and why of threads

Purpose of threads: To allow multiple parallel (simultaneous) paths of execution within a single program. Normally, in a running program, we have a single path of execution or thread.

The following are some examples of why threads are useful:

1. **To simplify design.** Example, a database server. Example, sending a heartbeat message on a TCP/IP pipe at fixed intervals. (I will explain why this is useful.) Our lab that keeps multiple processes running would be a lot easier with threads. Why?
2. **To allow parallel operations.** Example, multiple copies of files over the internet. I will explain why can this be a gain in performance. Compile multiple .cpp files at the same time. **(Eclipse does this for us if requested. Go to properties - C++ Build - Behavior and select "Enable parallel build".** This is very important if you have a large body of code - not important for us.) Visual Studio does this automatically.

Another example is a stock history simulation tool that updates its data in real-time. For simulations I needed a pseudo clock so we did not have to wait for actual time to go by.

Another use is some of the newer C++ STL algorithms. These use multiple threads. If you want to use in the work place, please test performance before using.

3. **To take advantage of multiple processors.** (This will become a very important reason.) Example, neural networks. Example, optimizing a stock trading model by doing multiple simultaneous runs.
4. **Enforce fairness.** I will use a server example in class.

Aspects of Programming Using Threads

The following are some ideas that you must be aware of if you are writing programs with threads.

1. Care must be taken in using existing libraries. For example, if printf is in several threads, your process has the potential of aborting. printf uses malloc'ed memory. (malloc is the C version of new) Functions that are reentrant are thread safe. Such devices as semaphores can make a function thread safe. Care must be taken to avoid deadlocks. **For g++ use the -pthread (or -lpthread) flag for both compiler and linker. For Eclipse add pthreads to list of libraries going to <project>-<properties>-<C/C++ Build>-<settings>-<GCC C++ Linker>-<General> and check support for pthread.** This should tell the compiler to use thread safe libraries and the POSIX thread library. These flags are a little different for other compilers/IDEs. Look at the help page for the compiler in the IDE that you are using.

Note: Some functions cannot be made thread safe without great pain. Can you guess what kind of functions fit this category?

2. **Global variables are available to all threads.** Care must be taken to defend against unexpected side effects from two threads using the same global variable. Example "errno". Don't worry, UNIX protects this global variable. (How would you guess that they do this?) Must use semaphores, mutexes, or other techniques to defend against these problems.

3. **Each thread has its own stack.** The size of the stack can be specified when the thread is created. The default is usually 8 mb for Linux and 1 mb for windows. (The Linux size was a surprise for me.) Stack overflow can be a severe problem. With luck, your program will abort. Also, **the space used for all the stacks in a large multithreaded application can be an issue.**
4. **You may need a special debugger to follow threads.** gdb has facilities for tracing multiple threads. You can tell it to attach to different threads. It is still a problem debugging threads because of conditions that will not show up when you are stepping through the code in a debugger. **Better might be to make the program single threaded, if possible, to perform the initial debugging. Then make it multithreaded.**
5. **One can explicitly set the priority, scheduling method, stack size, etc. for threads. The stack size can be a big help.**

Thread System Calls

The approach I will use to present material on multithreaded programming is to present a subset of the POSIX thread calls that will be sufficient to implement most applications. We will then talk about how threads are implemented, aspects of writing programs that use multiple threads, and then later present additional thread calls to deal with these.

Basic system calls: Some man/help pages for various UNIX systems might will read slightly different for the prototypes. But, they will all say the same thing. Just different naming for parameters and their types.

pthread_create

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)
(void *), void *arg );
```

thread - The thread object created. (This is the wording that is used in the description. The thread object is not an object of a class.) This is a structure that contains information that defines the structure. This is sometimes called the thread ID.

attr - Thread attributes object that defines the characteristics of the thread being created. For POSIX threads NULL is used to specify that default attributes are to be used. We will use NULL for now. **The thread priority and stack size are examples of elements of an attribute.**

start_routine - Function executed as the new thread's start routine. This is where the created thread begins execution. The function is a void function with a single void *

argument. [How do we pass the address of a function?](#)

arg - Address value copied and passed to the thread's start routine. This is a way that the thread can receive information from the calling program. We can pass a pointer to anything with a void pointer.

Returns 0 if successful and -1 if error.

pthread_exit

```
void pthread_exit( void *value_ptr );
```

The address value_ptr is copied and returned to the caller of pthread_join. (Discussed later.) This is a way that the thread can pass information back to the code that scheduled it when it exits. Be careful not to pass information that can disappear. For example, don't pass the address of an automatic variable. Why? Often people used addresses that have been allocated using malloc or new. They also use this return value to return a constant. How? Note: if you want to terminate a thread in its starting function, you can do so with a return call. (Note: the return value from a thread is a void pointer.) [Why not terminate a thread with an exit call?](#)

pthread_join

```
int pthread_join( pthread_t thread, void **value_ptr );
```

thread - Thread whose termination is awaited by the caller of this routine. Interesting that this is a call by value. (not an issue in that the thread ID is small)

value_ptr - Status value of the terminating thread (in other words, when that thread calls pthread_exit.) The value of this argument is NULL if you do not wish to look at the value returned from the exit. You fail an intelligence test if you call this function in the same thread that is specified by thread structure.

Returns 0 if successful and -1 if error. Note: this function returns immediately if the thread specified has already terminated.

[Example 1](#): thread1.cpp - demonstrates basic mechanics of creating threads. Note what would happen if we do not use an array of buffers. [Show what would happen if we forget the -pthread option. In eclipse go to <properties>-<C/C++ Build>-<Settings>-<GCC C++ Linker>-<general> and check support for pthread.](#) You can step through the code in the debugger and show how we can follow multiple threads. It is not easy. How do we debug a multithreaded program? Show what happens if the main terminates. Show what happens if we pass a single array of characters to all threads.

Example2: thread2.cpp - demonstrates the need for coordination among threads that shared the same global variables. Why volatile? Note: no need for this if you compiled only for debug. Let's compile for release and show the difference and without having this. What this difference? Run without the volatile keyword. Note the increase in speed and that we got the "correct" result. (Not seeing this now - we will talk about it). To compiled for release click on **<project> - <C/C++ Build> <Builder Settings>** and then select Debug or release and hit **<set Active>**.

Example3: thread3.cpp - shows how certain library calls can get messed up by thread operation. We are seeing how one buffer is used for all printf's and couts. I found this disturbing at first. What happened???? On other systems, I have seen the program blow up.

Example 4: thread4.cpp - demonstrates that all threads are dependent of the main thread for their existence. This is different than the .NET environment. What if one of the threads does an exit? Let's try this. Also show that a thread can't call exit.

Don't forget to add pthread to your libraries in Eclipse and -pthread as a command line argument for g++. We will talk more in class.

Comments and Reminders:

1. Global variables are available to all threads. See example 2 - thread2 shows how three threads can all access the same global variable. It also demonstrates the needs for a protection for two threads simultaneously accessing the same variable. Show how you get different results if this program is run twice.
2. Certain functions can get messed up by running on multiple threads. On some systems sleep used to get messed up things. See example 3 - thread3 for a demonstration of how printf and cout messes up. Since sleep is a system call and not associated with the thread library, things do not always work the way the we would hope. There are a bunch of system calls that have been rewritten to support threads. Select is an example of such a call.
3. How do you know which functions are designed for threads? The man or help pages will give you warnings. For example, lets look at **strtok**. See <https://en.cppreference.com/w/c/string/byte/strtok>
4. All functions that are called by more than one thread, must be reentrant. What does this mean? Why?
5. Solaris and some other UNIX systems still have their own implementation of threads in addition to POSIX threads. Why?? - this was their first implementation. So they keep it so that users do not get angry. They are very compatible with POSIX threads. You should use POSIX threads. This is not as big a deal as it once was.

6. It is difficult to launch a thread whose startup function lives in a class. Why? We will discuss how to do this in class.

Thread Synchronization

In order to prevent race conditions we need the ability to lock certain sections of code so that only one thread can execute them at a time. POSIX supplies a very simple and efficient tool to coordinate access to shared resource among threads. [These are called mutex locks \(or mutexes\)](#). Just like semaphores, mutex locks can only protect shared resources if all the threads that access the shared resource use them. Note: Mutexes can be made shared in order to control access among processes. This is done through the mutex attribute. See below.

Note: You must make sure that shared resources are protected. One way to be safe is to build these into classes with protection in the accessor functions and/or in properties.

Note: Locking and unlocking a mutex takes time. So, don't use them where they are not needed.

The following are the system calls that support mutex locks:

pthread_mutex_init

```
#include <pthread.h>
```

```
int pthread_mutex_init (pthread_mutex_t *mutex, pthread_mutexattr_t *attr)
```

mutex - Mutex created. This is set to the unlocked state.

attr - Mutex attributes object that defines the characteristics of the created mutex. In POSIX standards, NULL means use default attributes. You will probably always use this alternative. There is a `pthread_mutexattr_init` function to initialize the attributes. We will not worry about setting attributes for this course.

RETURNS 0 if successful and -1 if error.

NOTE: `PTHREAD_MUTEX_INITIALIZER` described in the text allows this value to be assigned to a mutex make initialize it in an unlocked state with the default attributes. If this is used, the `pthread_mutex_init` function is

unnecessary. (i.e. `pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;`)

Note: attributes are pretty special purpose tools to control access (e.g. Like spin locks) and we won't discuss them here. The function `pthread_mutex_init` could be used to initialize the attributes to the default values, but there is no need for this.

Note: The WIN32 version of mutexes are referred critical sections. These have additional functionality, but unfortunately, we cannot initialize them in as simple as manner as mutexes.

pthread_mutex_lock

```
#include <pthread.h>
```

```
int pthread_mutex_lock (pthread_mutex_t *mutex)
```

mutex - the address of a mutex.

- If the mutex is not locked, it is locked and execution continues.
- If the mutex is locked, the thread blocks until the mutex is unlocked.
- Be aware that recursion can cause a deadlocked state for a mutex. This is a very common problem. (Note: WIN32 critical sections do not have this problem. We will discuss why) [This function will return with -1 if a deadlock is detected.](#) Let's test this when we have an example.

Returns 0 if successful and -1 if error.

pthread_mutex_unlock

```
#include <pthread.h>
```

```
int pthread_mutex_unlock (pthread_mutex_t *mutex)
```

mutex - the mutex to be unlocked

- The mutex will be unlocked and if there is one or more threads waiting on the mutex, one of them will be released. Namely, it will return from the `pthread_mutex_lock` call that caused it to wait.

- Note: if the mutex is already unlocked, the results are unpredictable.

Returns: 0 if successful and -1 if fails.

pthread_mutex_trylock

```
#include <pthread.h>
```

```
int pthread_mutex_trylock (pthread_mutex_t *mutex)
```

mutex - the address of the mutex to be locked.

This function will locked the mutex if it is not locked. It will not wait for a locked mutex to become locked.

Returns: 1 if locked mutex, 0 if mutex was locked, and -1 if error.

[Example 5](#) thread5.cpp - demonstrates the use of mutex to avoid race conditions. It also demonstrates the speed of these calls. There is a cost associated with mutex. But, this is a better alternative to having the program perform incorrectly. [We will use the time command in class to time this function with and without using mutexes.](#) Note that the real-time is less than the user time. Why? (That might be unexpected) [Why are mutexes not making a bigger performance difference in this example?](#)

Yielding CPU time

POSIX provides a tool that allows a thread to yield the processor that is executing it to other threads of the same or greater priority. In POSIX the name of this function is sched_yield.:

```
#include <pthread.h>
```

```
void sched_yield();
```

You will notice that it is a very simple function with no arguments and no return value. Question: how would we use this method in Linux? Note library mentioned below.

[Example 6](#) thread6.cpp- demonstrates the use of sched_yield to allow another thread to run. Note: if you use a makefile, you will need **to specify the rt library with the -lrt parameter** in the make file. How did I find out about this parameter? [The rt library needs to be added to the libraries for Eclipse.](#) (Not needed anymore if you enable support for pthread.) Note that things don't always work the way that we would expect.

Why is the output not more disciplined? The reason I presented this is as an example of where something that looks useful might not be.

Run without sched_yield and see what happens. (Not much difference unless all processors are busy.) If we want to make our output really safe, how can we do it?

Note to self - add section on read and write locks another year

Lab12

Write a program to generate 100,000 prime numbers. You will write this program under the assumption that the program will be run on a computer with 5 CPUs and that your goal is to have this program finish as quickly as possible. You should store the primes in a file which is a command-line argument. We will discuss how to do this program in class.

An Alternative To Mutexes

Considering the following link: <http://gcc.gnu.org/onlinedocs/gcc-4.1.2/gcc/Atomic-Builtins.html> This describes a GNU alternative to mutexes. It is based on the compare and swap machine language instruction. I will discuss it in class. It provides a very fast locking mechanism, but, depending on use, can be CPU intensive. We will try: **bool __sync_bool_compare_and_swap (type *ptr, type oldval type newval, ...)** in class. Compares *ptr to oldval. If equal, records new value and returns true. This allows us to create a spin lock. We will implement a spin lock in class and time it. (See thread5.cpp example.) It should be interesting to answer questions that arise. There are other approaches that are less complicated.

Conditional Variables

Conditional variables are used to allow a thread to wait on an arbitrary predicate. This is in contrast to mutexes that allow a thread to wait until the mutex is unlocked. This concept is vital to quite a number of uses for threads.

Conditional variables are difficult to understand. So, expect things to be a little blurry at first. Hopefully my example will clarify.

Example of the need

Suppose we wanted to execute some code under mutex protection given an arbitrary predicate is true. (An example of such a predicate would be to test to see if there is data at the end of a queue.) At first glance it might seem like the following would be an adequate implementations:

1. Create and initialize a mutex.
2. Acquire the mutex.
3. Test the predicate. If it is true, execute the indicated code.
4. In either case, release the mutex.
5. Go to step 2.

What is wrong with this? At first glance, nothing. But the big problem is that we use a lot of CPU time waiting for the predicate to be true. This is especially the case if there are multiple threads waiting for the predicate to be true. There might be even more time than you realize if the function that changes data reference by the predicate has trouble locking the mutex. What we need is a way of signaling to the threads that they should test the predicate. This is where the conditional variables come into play. They are used to wake up the thread when the predicate has potentially a new value.

As mentioned previously, an ideal application is queues.

The new flow of execution will require a mutex and a conditional variable. The flow goes as follows:

1. create a mutex and a conditional variable.
2. Acquire the mutex.
3. As long as the predicate is not true do the following.
 - a) Atomically release the mutex
 - b) Wait for the conditional variable to be signaled that there is a potential change in the predicate.
 - c) When the conditional variable is signaled, automatically acquire the mutex and test the predicate. If the predicate is not true, go to step a.
4. Do what ever you want.
5. Release the mutex.
6. Go to 2

pthread_cond_init

The following are a minimal set of system calls to support conditional variables. See the text for a complete set. Note: you can really appreciate the literacy (or lack thereof) of the mutex programmers by the quality of the manual pages.

```
#include <pthread.h>
```

```
int pthread_cond_init (pthread_cond_t *cond, pthread_condattr_t attr )
```

As the name implies, this function initializes a conditional variable.

cond - Condition variable that is created.

attr - Condition variable attributes object that defines the characteristics of the condition variable created. NULL is used indicate default attributes.

Returns 0 if successful and -1 if fails.

Note: In a similar manner as we did with mutexes, we can initialize a conditional variable by:

```
pthread_cond_t condition = PTHREAD_COND_INITIALIZER;
```

pthread_cond_wait

```
#include <pthread.h>
```

```
int pthread_cond_wait ( pthread_cond_t *cond, pthread_mutex_t *mutex)
```

cond - Condition variable waited on.

mutex - Mutex associated with the condition variable specified in cond.

This function atomically releases the mutex and wait for the conditional variable to be signaled or broadcasted. When this occurs, the mutex is reacquired (i.e. locked) and the function returns. The return value is 0 if this function is successful and -1 if it fails.

pthread_cond_broadcast

```
#include <pthread.h>
```

```
int pthread_cond_broadcast ( pthread_cond_t *cond )
```

cond - Condition variable broadcasted to.

This function wakes all threads waiting on a condition variable. (This will be one at a time when they can acquire the mutex.) This function has no effect if there are no threads blocked on the conditional variable.

Returns 0 if successful and -1 if error.

pthread_cond_signal

```
#include <pthread.h>
```

```
int pthread_cond_signal ( pthread_cond_t *cond; )
```

cond Condition variable signaled.

This function will wake up one of the threads ("at least one of the threads" according to the man page) waiting on a condition variable.

Returns 0 if successful and -1 if fails.

See example 8 [thread8](#) to show how these functions work. It will make sure that 5 threads execute in a specific order. The predicate is indicating whose turn it is. Note that we need to broadcast rather than wake a single thread. We will discuss and demonstrate this.

We will talk about how this is a useful tool to implement queues.

Spin Locks - Hopefully, we talked about this previously.

See [SpinLock.cpp](#) for an example of use. It is based on the following code:

```
volatile int spinLockVar = 0;
```

```
void SpinLock()
```

```
{
```

```
    while( ! __sync_bool_compare_and_swap ( &spinLockVar, 0, 1 ) ); // Wait for  
    unlocked.
```

```
    return;
```

```
}
```

Final Note

We will talk a little more in class about thread pools. Easy to implement with queues and conditional variables.

Lab14 Extra Credit

Using conditional variables and mutexes, write program to create a queue class that will allow multiple threads to read and write from the queue. We will discuss how to do this in class.